Examen final
Personaje para ser expuesto en Three.js usando el casco MetaQuest 3 512

Guía de Verificación Técnica para Modelado en Blender (WebXR)

1. Nombre y Ruta de Exportación del Archivo

- **Línea de código en la que impacta:**

  'models/gltf/Soldier.glb',

  Solución en Blender: Al exportar el modelo final en formato .glb, el alumno debe **guardarlo con el nombre exacto de Soldier.glb** (**respetando la "S" mayúscula**) y ubicarlo manualmente dentro del directorio del proyecto bajo la ruta exacta de carpetas models/gltf/.

- **¿Por qué se debe hacer así?** El cargador asíncrono de Three.js (GLTFLoader) tiene una ruta de acceso rígida programada por defecto. Si el archivo se nombra de otra manera o varía por una sola letra, el servidor responderá con un error HTTP 404 (File Not Found), provocando una promesa rechazada que detendrá la inicialización gráfica.

2. Orden Secuencial de las Pistas de Animación (Índices)

Líneas de código en las que impacta:

Idle: mixer.clipAction(anims[0]),
Run: mixer.clipAction(anims[1]),
TPose: mixer.clipAction(anims[2])

Solución en Blender: En el editor de animación (Non-Linear Animation o Action Editor), el alumno debe renombrar forzosamente cada clip de animación anteponiendo un número para manipular el orden alfabético: "0_Idle" (reposo), "1_Run" (carrera) y "2_TPose" (vuelo con brazos en T).

- **¿Por qué se debe hacer así?** El arreglo nativo anims guarda los clips en el orden físico secuencial que genera el exportador de Blender (el cual lee alfabéticamente). El programa no busca las animaciones por su etiqueta de texto, sino por su índice en memoria (0, 1 o 2). Si se altera este orden, las animaciones del mando se cruzarán de forma errónea.

3. Rotación Inicial del Modelo y Conversión del Eje Vertical

Líneas de código en las que impacta:

```
model.rotation.y = PI;
group.rotation.y = PI;
```

Solución en Blender: El personaje debe modelarse y animarse mirando hacia el frente en la vista frontal local de Blender (Eje -Y hacia adelante). Al momento de exportar a glTF 2.0, en la pestaña lateral derecha de transformaciones, se debe verificar que la opción "Transform: +Y Up" se encuentre marcada.

¿Por qué se debe hacer así? Blender utiliza un sistema de coordenadas donde el eje Z es la vertical, mientras que WebGL/Three.js utiliza el eje Y como altura. Activar "+Y Up" obliga al exportador a realizar la conversión espacial automática. Además, el programa rota el modelo 180 grados (PI) al cargar para alinearlo de cara a la cámara inicial de las Quest; si no se diseña con el frente correcto, el personaje aparecerá dándole la espalda al usuario.

4. Cadencia de Animación y Sincronización Estética

Líneas de código en las que impacta

```
runVelocity: 5,
// ...
const vel = (controls.current === 'TPose') ? 6.0 :
controls.runVelocity;
```

Solución en Blender: En la animación de carrera (1_Run), el alumno debe **ajustar la cantidad de fotogramas clave (keyframes)** y la velocidad de zancada para que sea enérgica e intensa, coordinada para cubrir una distancia visual representativa en su loop.

- **¿Por qué se debe hacer así?** El motor matemático del programa desplaza la matriz de posición del grupo tridimensional a una velocidad fija de 5 unidades por segundo a través del espacio virtual. Si la animación en Blender está diseñada con pasos lentos o pausados, se romperá la coherencia visual, provocando un efecto de deslizamiento o "patinaje sobre hielo" del personaje sobre la superficie del suelo.

5. Escala Global Nativa del Archivo (Transformaciones)

Línea de código en la que impacta
alturaObjetivo = camera.position.y + 4.0;

Solución en Blender: El personaje debe modelarse respetando las proporciones humanas reales utilizando la cuadrícula métrica de Blender (ej. entre 1.70 y 1.85 metros de altura total). Antes de exportar, en el modo objeto (Object Mode), se debe seleccionar el esqueleto y la malla, presionar Ctrl + A y elegir la opción "**All Transforms**" para fijar la escala nativa en 1.0.

¿Por qué se debe hacer así? El código no escala al personaje artificialmente mediante software; consume directamente las dimensiones nativas que hereda el archivo tridimensional. Al iniciar el vuelo, el programa calcula elevar al personaje exactamente 4 metros sobre la posición actual de la cámara virtual. Si el modelo fue escalado de forma incorrecta en Blender sin aplicar sus transformaciones, el cálculo matemático se desfasará por completo en el visor VR.

6. Ubicación del Punto de Origen (Punto de Pivote Central)

Línea de código en la que impacta

position.y += (alturaObjetivo - position.y) * 4.0 * delta;

Solución en Blender: El punto de origen del objeto (representado por **el punto naranja central tanto en el esqueleto Armature como en las geometrías de la malla) debe estar ubicado matemáticamente en la coordenada de origen absoluto de Blender (0, 0, 0),** colocado exactamente a nivel del suelo, centrado en medio de ambos pies del personaje.

¿Por qué se debe hacer así? La variable de traslación position.y calcula la altura del personaje tomando como referencia matemática su punto pivote de origen. Si el alumno deja el pivote por defecto en el centro del pecho o en el ombligo del modelo, el código interpretará que esa área central es la base del personaje, lo que provocará que el soldado aparezca enterrado hasta la cintura dentro del piso con textura de ajedrez.

7. Estructura de Rigging y Jerarquía de Huesos

Líneas de código en las que impacta

let group, model, skeleton; // ... panel.add(settings, 'show_skeleton');

Solución en Blender: La deformación de la geometría debe estar controlada por un objeto de **tipo esqueleto estándar (Armature), y cada sección de la malla poligonal debe estar emparentada correctamente mediante influencias de pesos (Vertex Weights) bien pintadas sobre la malla**.

¿Por qué se debe hacer así? La aplicación extrae de forma automática la jerarquía ósea del modelo durante el proceso de inicialización para integrarlo a los controladores y a la interfaz gráfica de depuración en tiempo real del visor VR. Si el alumno exporta un personaje compuesto por piezas rígidas sueltas sin un sistema de huesos integrado, la referencia interna del objeto skeleton quedará indefinida y romperá de forma permanente el menú de herramientas del panel.

8. Obligatoriedad de Uso de Mallas Poligonales

Líneas de código en las que impacta

```
model.traverse(function(obj) {
    if (obj.isMesh) {
        obj.castShadow = true;
        obj.receiveShadow = true;
    }
});
```

Solución en Blender: Todo elemento geométrico que forme parte del personaje (cuerpo, ropa, cabello, accesorios) **debe ser estrictamente una malla de polígonos. Si se usaron curvas de Bézier, texto plano o superficies de tipo NURBS durante el modelado, se deben seleccionar y presionar clic derecho: Convert to > Mesh antes de exportar**.

¿Por qué se debe hacer así? La función de recorrido cíclico traverse inspecciona cada elemento dentro del archivo y evalúa mediante una condicional booleana si el elemento en cuestión es una malla (isMesh). Si resulta verdadera, le asigna los métodos de renderizado para proyectar y recibir sombras físicas. Cualquier objeto que no se convierta a malla evadirá esta instrucción y no proyectará sombras.

9. Sombreado y Materiales bajo el Estándar Físico PBR

Líneas de código en las que impacta:

renderer.toneMapping = THREE.ACESFilmicToneMapping;
renderer.shadowMap.type = THREE.PCFShadowMap;
scene.environmentIntensity = 1.5;

Solución en Blender: Los materiales aplicados al personaje deben construirse de forma exclusiva utilizando el nodo de sombreado estándar Principled BSDF dentro del editor de sombreadores (Shader Editor). Las imágenes de texturas (Color, Roughness, Normals) deben conectarse a sus respectivos canales en este nodo de Blender.

¿Por qué se debe hacer así? El motor de renderizado de la aplicación está configurado para calcular la luz mediante algoritmos PBR basados en física real, aplicando una corrección de rango dinámico cinematográfico (ACESFilmicToneMapping) de alta calidad. El nodo Principled BSDF se traduce directamente a este formato nativo tridimensional. **Si el alumno utiliza Shaders procedimentales avanzados propios de Blender o nodos de tipo cel-shading, el personaje se renderizará de forma incorrecta, totalmente negro o invisible**.

10 Configuración del Espacio de Color de las Texturas de Albedo

Líneas de código en las que impacta:

```
 renderer.toneMapping = THREE.ACESFilmicToneMapping;
renderer.toneMappingExposure = 0.5;
```

Solución en Blender: En el editor de sombreadores, la textura de imagen que se conecta al canal Base Color del material **debe tener su propiedad de espacio de color configurada explícitamente en sRGB**. **Los mapas de datos matemáticos (como Normal Maps o Roughness) deben configurarse como Non-Color**.

¿Por qué se debe hacer así? Al tener una exposición de luz configurada de forma fija en un nivel de 0.5, el renderizador **exige que los datos cromáticos de entrada lleguen corregidos con la curva gamma adecuada**. S**i el alumno configura accidentalmente la textura de color del personaje como Non-Color o Raw, los tonos se procesarán de forma lineal, provocando que bajo el visor de Meta Quest 3 el soldado se vea con colores extremadamente lavados, quemados o pálidos**.

11 Jerarquía y Colapso de la Pila de Modificadores

Líneas de código en las que impacta:

```
 model.quaternion.rotateTowards(rotate, controls.rotateSpeed *
2); // ... if (mixer) mixer.update(delta);
```

Solución en Blender: En la pestaña de modificadores del personaje, el modificador Armature debe estar posicionado estrictamente en el **primer lugar de la lista (hasta arriba)**. Cualquier otro modificador estructural utilizado durante el desarrollo (como Mirror para simetría o Subdivision Surface) debe ser aplicado y colapsado permanentemente sobre la malla polimórfica antes de exportar el archivo .glb.
¿Por qué se debe hacer así? El mezclador de animaciones (mixer) calcula las transformaciones de posición de los vértices cuadro por cuadro de forma dinámica. **Si se deja un modificador como Mirror activo sin aplicar, la geometría dependerá de cálculos internos de Blender que no existen en Three.js**, lo que causará errores graves de desmembramiento visual del personaje al girar o desplazarse.

12 Desplegado y Coordenadas de Mapeo UV Limpias (Sombras)

Línea de código en la que impacta:

obj.receiveShadow = true;

Solución en Blender: El alumno debe realizar un proceso completo y limpio de desplegado de mapas UV (UV Unwrapping) sobre toda la geometría del personaje, **garantizando que los polígonos no se empalmen de forma desordenada en el lienzo bidimensional y que cuente con un único mapa UV primario activo**.

¿**Por qué se debe hacer así**? Para que el algoritmo de sombras pueda proyectar las sombras dinámicas sobre el relieve del cuerpo del soldado de manera correcta, requiere leer las coordenadas de mapeo UV de la geometría para determinar la profundidad espacial de los polígonos. **Si la malla no posee mapas UV o estos se encuentran corruptos o encimados, el cálculo matemático fallará, provocando parpadeos negros sobre la piel del personaje**.

13 Orientación de Caras de la Malla (Face Orientation)

Líneas de código en las que impacta:

obj.castShadow = true;
obj.receiveShadow = true;

Solución en Blender: El alumno debe verificar que la dirección de las normales de los polígonos apunte hacia afuera. En el visor 3D de Blender, debe desplegar el menú de Overlays y **activar la casilla "Face Orientation"**. **Toda la superficie del personaje debe verse de color azul. Si alguna parte se ve roja, debe entrar a Edit Mode, seleccionar todo (A), presionar Alt + N y elegir "Recalculate Outside"**.

¿Por qué se debe hacer así? WebGL renderiza las superficies utilizando una técnica de optimización llamada Back-face Culling, lo que significa que **solo dibuja las caras que miran hacia la cámara para ahorrar procesamiento**. Si el alumno deja caras invertidas (rojas en Blender), esas partes del cuerpo del personaje se volverán completamente invisibles en las Meta Quest 3 y romperán el cálculo de iluminación.

14 Influencia Máxima de Huesos por Vértice (Deformación de Malla)

Líneas de código en las que impacta:

if (mixer) mixer.update(delta); // ... updateCharacter(delta);

Solución en Blender: Durante el proceso de pintado de pesos (Weight Painting), el alumno debe asegurarse de que un solo vértice de la malla no esté influenciado por más de 4 huesos simultáneamente. Al exportar a .glb, dentro de la pestaña Animation, debe verificar que la opción de optimización de límites de huesos por vértice esté activa (fijada en un máximo de 4).

¿Por qué se debe hacer así? Las GPU de dispositivos móviles como las Meta Quest 3 procesan la animación de personajes (Skinning) usando sombreadores de hardware optimizados. **Three.js limita por rendimiento el estándar a un máximo de 4 articulaciones por vértice para mantener la tasa de refresco fluida**. Si un vértice es controlado por 5 o más huesos, la tarjeta gráfica del visor VR ignorará los pesos excedentes o deformará la malla de forma caótica.

15 Vinculación del Esqueleto y el Nodo Raíz (Armature Parent)

Líneas de código en las que impacta:

scene.fog = new THREE.Fog(0x5e5d5d, 2, 20); // ...
group.position.copy(position);

Solución en Blender: En la jerarquía del Outliner de Blender, la malla poligonal del personaje debe estar estrictamente emparentada debajo del objeto Armature (el esqueleto debe ser el padre directo). Además, ni la malla ni el esqueleto deben estar metidos dentro de carpetas vacías (Empty objects) secundarias que tengan transformaciones de posición o rotación diferentes a cero.

¿Por qué se debe hacer así? Tu aplicación **calcula un efecto de niebla lineal muy cerrado, que va desde los 2 metros hasta los 20 metros de distancia máxima desde la cámara**. Si el alumno deja un nodo vacío intermedio o no emparenta la malla al esqueleto correctamente, Blender exportará un nodo raíz artificial descentrado. Esto provocará que cuando la línea group.position.copy(position) mueva al soldado por el mapa, las coordenadas del esqueleto se desfasen de la malla real y el personaje desaparezca de inmediato dentro de la niebla gris.

16 Optimización de Texturas: Dimensiones en Potencias de 2

Líneas de código en las que impacta:

renderer.setPixelRatio(window.devicePixelRatio);
renderer.setSize(window.innerWidth, window.innerHeight);

Solución en Blender: Todas las texturas de imagen externas que el alumno cargue dentro del nodo Principled BSDF (Color, Normal, Rugosidad) deben tener resoluciones con dimensiones que sean potencias de dos (ej. 1024x1024, 2048x2048 píxeles). Bajo ninguna circunstancia deben usarse resoluciones arbitrarias como 1200x850 o 3000x2000. El tamaño máximo recomendado para WebXR móvil es 2K

¿Por qué se debe hacer así? Al empaquetar y renderizar en la resolución nativa de las Quest 3, la GPU utiliza una técnica llamada **Mipmapping (crear copias más chicas de la textura para objetos lejanos)**. **Si la textura no está en potencia de dos, el motor gráfico de Three.js se ve obligado a redimensionar la imagen en tiempo real mediante CPU antes de enviarla a la GPU. Esto satura la memoria intermedia y genera tirones de procesamiento (stuttering) en VR**.

17 Presupuesto Poligonal Máximo para WebXR Móvil

Línea de código en la que impacta:

renderer.render(scene, camera);

Solución en Blender: El alumno debe controlar estrictamente la densidad de la malla. El modelo final del personaje (incluyendo accesorios) no debe superar los 15,000 a 20,000 polígonos (triángulos). Para verificarlo, el alumno debe activar la casilla "Statistics" en los Overlays de Blender. Si el diseño es muy complejo, debe usar el modificador Decimate para reducir polígonos antes de colapsar la malla.

¿Por qué se debe hacer así? El bucle de animación se ejecuta dentro del ciclo nativo de refresco de las Meta Quest 3, el cual exige pintar la escena a altas tasas de fotogramas por segundo fijos divididos en dos pantallas (una para cada ojo). Si el alumno exporta un personaje hiperrealista con cientos de miles de polígonos, la GPU móvil no podrá procesar la función de renderizado a tiempo, lo que causará una caída severa de FPS y provocará náuseas instantáneas en el usuario.

18 Nombres Únicos de Materiales y Eliminación de Bloques Huérfanos

window.addEventListener('unhandledrejection', function(event) { window.trace("PROMESA_RECHAZADA", event.reason); });

Solución en Blender: El alumno debe nombrar de forma única el material del personaje (ej. Mat_Soldado) y purgar los materiales antiguos que ya no use. Antes de guardar el archivo, debe ir al menú superior: File > Clean Up > Purge Unused Blocks para eliminar cualquier rastro de texturas o materiales huérfanos que hayan quedado flotando en la memoria del archivo .blend.

¿Por qué se debe hacer así? Cuando el exportador genera el binario .glb, si detecta que la malla tiene asignados "slots" de materiales vacíos, nombres duplicados por defecto (como Material.001, Material.002) o referencias rotas a texturas borradas, creará nodos corruptos en la estructura JSON del archivo. Al intentar parsearlo en el navegador, el motor disparará un fallo de lectura asíncrona que será atrapado por tu interceptor global unhandledrejection, enviando alertas de error consecutivas a tu servidor FastAPI.

19 Interpolación de Animaciones: Animaciones Lineales sin Remuestreo Complejo

Líneas de código en las que impacta:

curAct.reset();

curAct.setEffectiveWeight(1.0);

oldAct.fadeOut(fade);

curAct.fadeIn(fade).play();

Solución en Blender: Al animar los huesos del personaje, el alumno debe utilizar única y e**xclusivamente la interpolación de fotogramas clave en modo Linear (Lineal) o Bezier estándar para la rotación y posición**. Al exportar a .glb, dentro de la pestaña Animation, debe asegurarse de dejar desactivada la opción **"Always Sample Animations"** (Siempre remuestrear animaciones).

¿Por qué se debe hacer así? Tu código utiliza los métodos nativos .fadeIn() y .fadeOut() de Three.js para mezclar de manera matemática las animaciones en tiempo real (por ejemplo, pasar suavemente de Idle a Run). Si el alumno exporta animaciones sobrecargadas de fotogramas redundantemente remuestreados, el mezclador de animaciones saturará el hilo principal de procesamiento de la CPU de las Meta Quest 3, provocando saltos bruscos o deformaciones en la malla durante las transiciones suaves.

20 El Cálculo de la Caja de Cajas (Bounding Box / Bounding Sphere) de la Malla

Líneas de código en las que impacta:

```
alturaObjetivo = camera.position.y + 4.0;
// ...
position.y += (alturaObjetivo - position.y) * 4.0 * delta;
 // ...
group.position.copy(position);
```

Solución en Blender: El alumno debe asegurarse de que todas las mallas del personaje tengan sus transformaciones completamente limpias y que ningún fotograma de la animación desplace al esqueleto o a la malla lejos de su punto de pivote original en Blender. Toda la traslación (el movimiento de avanzar o subir) **debe ser de tipo In-Place (sin Root Motion)**.

¿Por qué se debe hacer así? El motor gráfico de las Quest 3 utiliza una técnica de optimización llamada Frustum Culling (Descarte por Frustum). Para ahorrar recursos, el visor calcula una esfera invisible alrededor del modelo llamada Bounding Sphere. Si la animación en Blender desplaza el modelo lejos de su centro físico, en cuanto el código ejecute la elevación de altura del grupo, el visor interpretará que el personaje ha salido del campo de visión real y dejará de renderizar al personaje de forma repentina.

21 El Uso de Múltiples Materiales Encimados y la Opacidad (Material Slots)

Línea de código en la que impacta:

```
const mat = new THREE.MeshStandardMaterial({
    map: floorT,
    color: 0x404040,
    depthWrite: false,
    roughness: 0.85
  });
```

Solución en Blender: El alumno debe asegurarse de que la malla del personaje utilice la menor cantidad de ranuras de material (Material Slots) posibles (**lo ideal es un solo material para todo el cuerpo)**. Además, **debe asegurarse de que el modo de mezcla (Blend Mode) en las propiedades del material de Blender esté configurado estrictamente como Opaque (Opaco), evitando Alpha Blend o Alpha Hashed si el diseño no requiere transparencias reales**.

¿Por qué se debe hacer así? Tu código desactiva la escritura de profundidad (depthWrite: false) en el suelo por optimización. Si el alumno exporta un personaje con materiales configurados con transparencias por error, T**hree.js perderá la capacidad de ordenar correctamente la profundidad de los polígonos**. En las pantallas de las Meta Quest 3, esto provocará que partes internas o traseras del

modelo se transparenten de forma errática y se superpongan de manera incoherente sobre la piel exterior del personaje.